

Tribhuvan University
Institute of Science and Technology



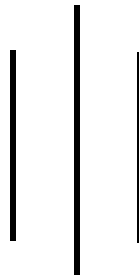
A Term Paper

On

**“Evaluation and Context Optimization of Retrieval-Augmented Generation for
Document Question Answering”**

Submitted To

Central Department of Computer Science and Information Technology
Tribhuvan University, Kirtipur
Kathmandu, Nepal



Submitted By

Melina Giri
Roll No: 8125018

April, 2026

**In partial fulfillment of the requirement for Master’s Degree in Information
Technology (MIT), Third Semester**



Tribhuvan University
Institute of Science and Technology

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this term paper, prepared under the supervision of Asst. Prof. Balkrishna Subedi entitled "**Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering**" be accepted as fulfillment in partial requirement for the degree of Master's in Information Technology. To the best of my knowledge, this is an original work in computer science.

.....

Asst. Prof. Balkrishna Subedi

(Supervisor)

Central Department of Computer Science and Information Technology

LETTER OF APPROVAL

This is to certify that the term paper prepared by **Ms. Melina Giri** entitled "**Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering**" in partial fulfillment of the requirements for the degree of Masters in Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Evaluation Committee

.....
Asst. Prof. Dr. Dhiraj Kedar Pandey
(H.O.D)

Central Department of Computer Science and
Information Technology

.....
Asst. Prof. Balkrishna Subedi
(Supervisor)

Central Department of Computer Science and
Information Technology

.....
(Internal)

ACKNOWLEDGEMENT

The success and final outcome of this report required a lot of guidance and assistance from many people and I am very fortunate to have received it throughout the completion of this work. I am very glad to express my deepest sense of gratitude and sincere thanks to my highly respected and esteemed supervisor **Asst. Prof. Balkrishna Subedi**, Central Department of Computer Science and Information Technology, for his valuable supervision, guidance, encouragement, and support in completing this paper.

I am also thankful to **Asst. Prof. Dr. Dhiraj Kedar Pandey**, HOD of Central Department of Computer Science and Information Technology for his constant support throughout the period. Furthermore, with immense pleasure, I extend my deepest gratitude to the Department of Computer Science and Information Technology, Tribhuvan University, and all the faculty members of CDCSIT for providing the platform to explore the knowledge of interest.

I would also like to acknowledge the authors and contributors of the **Open RAG Benchmark** provided by Vectara. The publicly available benchmark dataset was fundamental for conducting the retrieval and generation experiments presented in this research.

At the end I would like to express my sincere thanks to all my friends and others who helped me directly or indirectly.

Melina Giri (8125018)

Central Department of Computer Science and Information Technology

ABSTRACT

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation to provide answers using relevant information retrieved from external document collections. This study implements and evaluates a **RAG-based document question-answering system** using the **Vectara Open RAG Benchmark**. The system loads the benchmark’s pre-segmented document sections directly, generates dense embeddings using the **all-MiniLM-L12-v2** model, and retrieves relevant sections using vector similarity. Retrieval performance was evaluated using **3,045 benchmark queries** at retrieval depths of $k=3, 5, 10$, and 20 using Hit@ k , Mean Reciprocal Rank (MRR), and normalized Discounted Cumulative Gain (nDCG). The results show that retrieval effectiveness improved with increasing retrieval depth, with Hit@20 reaching **82.33%**, compared with **61.18%** at $k=3$. Generation performance was evaluated using **60 questions** under three configurations: **R0_k3**, **R1_k10**, and **R2_k10_optimized**, using the locally executed **Qwen2.5-3B** language model through Ollama. Among the three configurations, **R1_k10** achieved the **highest mean lexical F1 score** of 0.3136, while **R2_k10_optimized** achieved the **lowest average generation latency** of 12.098 seconds. A paired Wilcoxon signed-rank test on the 60 per-question lexical F1 scores showed that the decrease in F1 from R1_k10 to R2_k10_optimized was statistically significant ($p = 0.005$), while the differences involving R0_k3 were not significant. The findings indicate that increasing retrieval depth improves retrieval coverage, and that context optimization measurably improves generation efficiency but comes at the cost of a statistically significant, if small, reduction in lexical answer overlap. The study also identifies the limitations of lexical F1 and recommends semantic and human-based evaluation methods for future research.

Keywords: Retrieval-Augmented Generation, Document Question Answering, Dense Retrieval, Sentence Embeddings, Vector Database, Context Optimization, Open RAG Benchmark, Qwen2.5.

Table of Contents

SUPERVISOR’S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Report Organization	3
CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS	4
2.1 Background Study	4
2.1.1 Limitations	4
2.2 Literature Review	5
CHAPTER 3: METHODOLOGY AND CRITICAL ANALYSIS	7
3.1. Methodology	7
3.1.1. Flowchart	7
3.1.2. Dataset Description	8
3.1.3. Retrieval Analysis	8
3.1.4. Algorithm Description	9
3.2. Implementation	11
3.2.1. Implementation Tools	11
3.2.2. Implementation Details	11
CHAPTER 4: RESULTS/ FINDINGS	13
4.1. Findings and Observations	13
4.2. Result Analysis	13
4.2.1. Quantitative Analysis	13
4.2.2. Qualitative Analysis (Actual System Output)	17
CHAPTER 5: CONCLUSION	19
5.1. Conclusion	19
5.2. Future Recommendation	20

References	22
Appendix	23
A.1 Main Interface Code	23
A.2 Loading Documents and Corpus Sections	24
A.3 Embedding Generation and Vector Retrieval	25
A.4 RAG Answer Generation Code	26
A.5 Evaluation and Result Analysis Code	27

LIST OF FIGURES

Figure 3.1 : Overview of RAG-Based Document Question-Answering Workflow	7
Figure 4.1.1 : Retrieval Performance at Different Retrieval Depths	14
Figure 4.1.2 : Lexical F1 Comparison of RAG Generation Configurations	15
Figure 4.1.3 : Average Generation Latency Across RAG Configurations	15
Figure 4.1.4 : Sample Question and Generated Answer from the RAG System	17
Figure 4.1.5 : RAG Pipeline	18

LIST OF TABLES

Table 4.1 : Retrieval Performance at Different Retrieval Depths	13
Table 4.1.2 : Generation Performance of Different RAG Configurations	16
Table 4.1.3 : Paired Lexical-F1 Differences and Wilcoxon Signed-Rank Test Results	16

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
F1	F1 Score
IDK	I Don't Know
k	Retrieval Depth / Number of Retrieved Sections
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	normalized Discounted Cumulative Gain
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid growth of digital documents and research publications has created a significant challenge in efficiently finding and extracting relevant information. Traditional document search systems generally rely on keyword matching, which may return large numbers of documents without providing a direct, context-aware answer to the user's question. As the volume of digital information continues to increase, there is a growing need for systems that can retrieve relevant information and generate meaningful answers from that information.

Recent advances in **Large Language Models (LLMs)** have significantly improved natural language question-answering capabilities. However, language models may have limitations when answering knowledge-intensive questions that require information from external document collections. This limitation has motivated the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval** with **language generation**. Instead of relying only on information encoded in the language model, a RAG system first retrieves relevant sections from an external document collection and provides those sections as context to the language model. The model subsequently generates an answer using the retrieved context [1]. A typical RAG pipeline consists of document processing, section retrieval, embedding generation, vector-based retrieval, context selection, and answer generation [1].

The effectiveness of a RAG system depends not only on the quality of the language model but also on the quality and quantity of retrieved context. Retrieving too few sections may omit information necessary to answer a question, whereas retrieving too many sections may introduce redundant or irrelevant information and increase computational cost. Therefore, determining an appropriate retrieval depth and optimizing the retrieved context are important considerations in RAG-based document question answering.

This study evaluates a RAG system for document question answering using the **Open RAG Benchmark** [4], focusing on retrieval performance and the effect of different retrieval and context-optimization strategies. The study evaluates retrieval at different values of k and compares three generation conditions: retrieval of the top three sections, retrieval of the top ten sections, and retrieval of the top ten sections followed by context optimization. The study therefore investigates the relationship between retrieval depth, context optimization, answer quality, and generation latency.

1.2 Problem Statement

Although Retrieval-Augmented Generation systems can improve document-based question answering by supplying relevant external context to a language model, their performance is strongly influenced by the retrieval strategy and the amount of context provided to the model. Retrieving a small number of sections may fail to provide sufficient information for answering a question, while retrieving a larger number of sections may introduce redundant information and increase processing requirements.

Furthermore, simply increasing the number of retrieved sections does not necessarily guarantee better generated answers. Context selection and optimization may reduce unnecessary information and generation latency, but excessive removal of context may also eliminate information required for producing an accurate answer.

Therefore, there is a need to experimentally evaluate how **retrieval depth and context optimization affect RAG performance**, including whether observed differences in answer quality are statistically meaningful or attributable to chance. This study addresses this problem by evaluating retrieval effectiveness at different k values and comparing multiple generation configurations, including a paired significance test, using a standardized document question-answering benchmark.

1.3 Objectives

To evaluate the performance of a Retrieval-Augmented Generation (RAG) system for document question answering and analyze the effects of retrieval depth and context optimization on answer quality and generation efficiency.

The specific objectives of this study are:

- To implement a document question-answering pipeline using Retrieval-Augmented Generation, incorporating document processing, embedding generation, vector-based retrieval, and language-model-based answer generation.
- To evaluate and compare retrieval and generation performance at different retrieval depths and RAG configurations using Hit@ k , Mean Reciprocal Rank (MRR), nDCG, and lexical F1 metrics, including statistical significance testing.
- To analyze the effect of context optimization on answer quality and generation efficiency, particularly in terms of context size and generation latency, and identify the strengths and limitations of the evaluated RAG configurations.

1.4 Report Organization

This report is organized into five sections:

- **Section 1 (Introduction):** Presents an overview of the research problem, its significance, and the objectives of the study.
- **Section 2 (Background Study and Literature Review):** Reviews Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), dense retrieval, embeddings, and context optimization, along with relevant previous research.
- **Section 3 (Methodology and Implementation):** Describes the research methodology, dataset, RAG architecture, retrieval process, context optimization, evaluation metrics, and implementation details.
- **Section 4 (Results and Findings):** Presents the experimental results, retrieval and generation performance, statistical analysis, qualitative outputs, and key findings.
- **Section 5 (Conclusion and Future Work):** Summarizes the major findings and conclusions of the study and discusses limitations and possible directions for future work.
- **Appendix:** Contains supporting materials such as implementation code, RAG pipeline components, and evaluation code.

CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS

2.1 Background Study

The rapid growth of digital documents and research publications has created a significant challenge in efficiently finding and extracting relevant information. Traditional document retrieval systems mainly depend on keyword matching, which may require users to examine multiple documents before finding the information they need. As the volume of digital information continues to increase, effective document question-answering systems have become increasingly important.

Early document question-answering approaches were largely based on **keyword search, rule-based methods, and information retrieval techniques**. Although these approaches can identify documents containing specific terms, they may have difficulty understanding the semantic meaning of a question or producing a direct answer from multiple relevant passages.

The development of **Large Language Models (LLMs)** has significantly improved natural-language question answering. However, language models may have limitations when answers require information from specific external document collections. This limitation has led to the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval and language generation**. Relevant information is first retrieved from an external document collection and then supplied as context to a language model. The model uses this retrieved context to generate an answer to the user's question [1]. A typical RAG system includes document processing, embedding generation, vector storage, similarity-based retrieval, context selection, and answer generation.

The performance of a RAG system depends strongly on the quality and quantity of retrieved context. Retrieving too few sections may omit important information, while retrieving too many sections may introduce redundant information and increase computational requirements. Therefore, retrieval depth and context optimization are important factors in developing efficient document question-answering systems.

2.1.1 Limitations

This study has several limitations. First, the generation experiment used only 60 questions, compared with 3,045 for retrieval evaluation, limiting the statistical power of generation-level comparisons. Second, answer quality was assessed using lexical F1, which measures word overlap and does not capture semantic correctness. Third, the observed latency for R0_k3 (185.600 s) is substantially higher than expected relative to R1/R2 and may reflect a runtime

artifact rather than a true effect of context size; this was not independently isolated. Fourth, the benchmark consists of general research-paper documents rather than domain-specific material, so findings may not generalize to specialized domains without further evaluation.

2.2 Literature Review

The increasing availability of digital documents has made efficient document retrieval and question answering an important research area. Traditional information retrieval methods generally rely on lexical matching between queries and documents. Although effective for exact keyword searches, such methods may fail when the query and relevant document use different words with similar meanings.

The introduction of **dense vector representations and embeddings** has provided a semantic approach to information retrieval. Instead of matching only individual keywords, text can be converted into numerical vectors, allowing semantically similar queries and document sections to be identified using similarity measures [2][3]. **Cosine similarity** can then be used to measure the similarity between vector representations.

Recent developments in **Large Language Models** have further improved natural-language generation. However, LLM-only systems may suffer from hallucination, outdated knowledge, and lack of access to domain-specific documents. RAG addresses these limitations by retrieving external evidence before generating an answer. This approach provides the language model with retrieved document context during answer generation [1].

An important component of RAG performance is the **number of retrieved documents or sections**, commonly represented by k . A small k may provide insufficient context, whereas a large k may introduce irrelevant or repetitive information. Context optimization techniques can therefore be applied to remove redundant sections before generation.

RAG systems are commonly evaluated at two levels: **retrieval performance** and **generation performance**. Retrieval can be evaluated using measures such as **Hit@k**, **Mean Reciprocal Rank (MRR)**, and **normalized Discounted Cumulative Gain (nDCG)**. Generated answers can be evaluated using automatic metrics such as lexical F1, although lexical measures primarily evaluate word overlap and should not be interpreted as complete measures of semantic correctness. Where a metric is compared in aggregate across multiple system configurations on the same question set, a paired statistical test (e.g., the Wilcoxon signed-rank test) is appropriate for determining whether an observed difference reflects a systematic effect rather than sampling variation.

In conclusion, previous research establishes that effective RAG performance depends on both **retrieval quality and generation context**. This foundation supports the methodology adopted in

the present study, which evaluates different retrieval depths and compares context configurations with paired significance testing to investigate their effects on answer quality and generation efficiency.

CHAPTER 3: METHODOLOGY AND CRITICAL ANALYSIS

3.1. Methodology

3.1.1. Flowchart

The overall workflow of the proposed RAG-based document question-answering system consists of document preparation, embedding generation, vector retrieval, context selection, and answer generation.

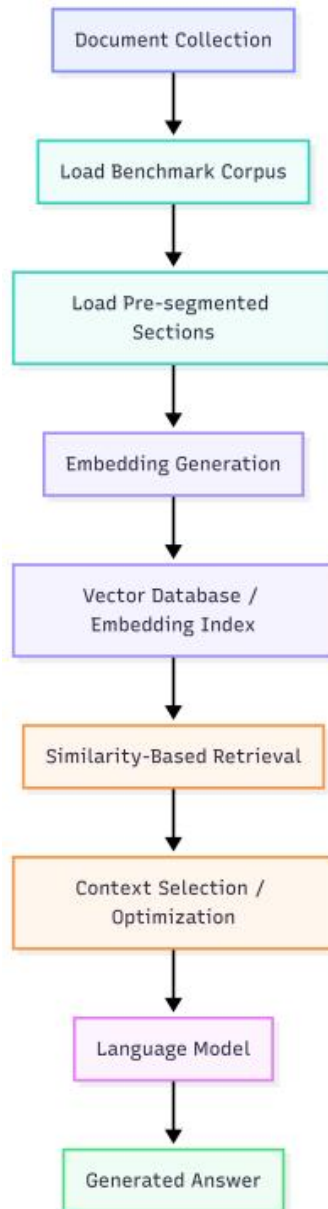


Figure 3.1: Overview of RAG-Based Document Question-Answering Workflow

The flowchart shows the main steps of the RAG-based question-answering process: document preprocessing, embedding generation, query processing, similarity-based retrieval, context selection/optimization, and answer generation.

3.1.2. Dataset Description

The dataset used for this study is the **Vectara Open RAG Benchmark**, which is designed for evaluating retrieval-augmented generation systems on research-paper documents. The benchmark contains **1,000 PDF papers and 3,045 valid question-answer pairs**. The benchmark includes both abstractive and extractive questions and contains text, text-image, text-table, and text-table-image source categories [4].

Data Processing

Before performing retrieval, the document corpus is loaded and transformed into searchable sections and numerical vector representations. The main processing steps include:

1. Document Preprocessing

- **PDF processing:** Each research paper in the Open RAG Benchmark corpus is provided as a structured JSON record containing document metadata and a list of pre-segmented sections.
- **Text segmentation:** The Open RAG Benchmark provides documents as pre-segmented sections. The study used these benchmark-provided sections directly as retrieval units, without applying additional token- or character-based splitting. Each section was retained together with its document and section identifier for subsequent embedding and retrieval evaluation.
- **Metadata preparation:** Document identifiers and section identifiers were retained to match retrieved sections against the benchmark’s relevance judgments.
- **Embedding generation:** Each document section was converted into a numerical vector representation.

2. Embedding Generation

- The study uses the **all-MiniLM-L12-v2** sentence-transformer model to generate dense embeddings for document sections and queries [3][6].
- Each document section is represented as a **384-dimensional vector**. These vectors allow semantic similarity to be calculated between a user query and **candidate document sections** [6].

3.1.3. Retrieval Analysis

The retrieval stage uses dense-vector similarity to identify the most relevant document sections for a given question, following the general principle of dense retrieval [2]. The query is converted into the same embedding space as the document sections, and similarity scores are calculated.

Different retrieval depths were evaluated, with $k \in \{3,5,10,20\}$. The retrieved sections were then compared against the benchmark's relevant sections.

Cosine Similarity (Mathematical Formula)

The similarity between a query vector q and document vector d is calculated using:

$$\text{Similarity}(q,d) = \frac{q \cdot d}{\|q\| \cdot \|d\|} \quad (1)$$

Where:

- q = query embedding vector
- d = document-section embedding vector
- $q \cdot d$ = dot product between the two vectors
- $\|q\|$ = magnitude of query vector
- $\|d\|$ = magnitude of document vector

Retrieval Evaluation Metrics

- **Hit@k:** Measures whether at least one relevant document section appears within the top k retrieved results.

$$\text{Hit@k} = \begin{cases} 1, & \text{if at least one relevant section is in top } k \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

- **Mean Reciprocal Rank (MRR):** Measures how highly the first relevant result is ranked.

$$\text{MRR} = \frac{1}{N} \sum_{i=0}^N \frac{1}{\text{rank}_i} \quad (3)$$

where rank_i is the position of the first relevant retrieved section for query i .

- **nDCG:** Measures ranking quality while giving greater importance to highly ranked relevant results.

$$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}} \quad (4)$$

where DCG is the discounted cumulative gain and IDCG is the ideal DCG.

3.1.4. Algorithm Description

The algorithm aims to answer document-based questions using Retrieval-Augmented Generation (RAG) [1]. It involves loading pre-segmented document sections, generating embeddings, retrieving relevant sections, optionally optimizing the retrieved context, and generating an answer using a language model.

Algorithm 1: Retrieval-Augmented Generation for Document Question Answering

Input: Document corpus D , user question q , retrieval depth k

Output: Generated answer A

Steps:

1. Document Processing

- Load the document corpus.
- Load the pre-segmented document sections provided by the Open RAG Benchmark.
- Generate embeddings for each document section.

2. Query Processing and Retrieval

- Accept the question from the user.
- Generate an embedding representation of the question.
- Calculate similarity between the query embedding and document-section embeddings.
- Rank document sections according to similarity.
- Select the top- k sections.

3. Context Optimization (R2_k10_optimized only)

- Examine the top-10 retrieved sections in ranked order.
- Convert each section into a set of lowercased words.
- Calculate the Jaccard word-set overlap between the candidate section and each previously selected section: $\text{overlap} = |A \cap B| / |A \cup B|$.
- Mark the candidate section as redundant if its overlap with any previously selected section is ≥ 0.70 .
- Remove redundant sections and retain a maximum of five non-redundant sections.

4. Context Assembly

- Concatenate the selected sections and truncate the combined context to a maximum of 8,000 characters, applied uniformly across all three configurations (R0_k3, R1_k10, R2_k10_optimized).

5. Generate Answer

- Combine the question with the retrieved context.
- Provide the context and question to the language model.
- Generate the final answer.
- If the available context is insufficient, the system is instructed to respond that the information is unknown.

3.2. Implementation

3.2.1. Implementation Tools

The implementation was carried out using the tools below for document processing, retrieval, vector representation, and answer generation:

- Python 3.x – Main programming language
- FastAPI – Backend API framework
- Streamlit – User-interface/frontend framework
- LangChain – RAG pipeline and language-model integration
- Sentence Transformers – Generation of dense text embeddings (all-MiniLM-L12-v2)
- NumPy – Vector similarity computation for retrieval
- Ollama – Local language-model execution
- Qwen2.5:3B – Local language model used for the final generation experiment
- Pandas – Dataset processing and result analysis
- SciPy – Paired t-test and Wilcoxon signed-rank significance testing
- Matplotlib – Visualization of experimental results
- VS Code – Development environment
- Dataset: Vectara Open RAG Benchmark

3.2.2. Implementation Details

The RAG system converts document sections and user queries into dense numerical representations using the all-MiniLM-L12-v2 embedding model. Each representation contains 384 dimensions. Query and document vectors are compared using cosine similarity to identify the most relevant document sections.

The retrieval experiment was performed using 3,045 benchmark queries. Retrieval performance was evaluated at four different retrieval depths: $k=3$, 5, 10, and 20.

For generation evaluation, 60 questions were selected from the benchmark. Three retrieval-context configurations were evaluated:

- **R0_k3**: Top 3 retrieved sections were provided to the language model.
- **R1_k10**: Top 10 retrieved sections were provided to the language model.
- **R2_k10_optimized**: Top 10 sections were retrieved and redundant context was removed (Jaccard overlap ≥ 0.70 , maximum 5 sections retained) before generation.

In all three configurations, the assembled context was additionally truncated to a maximum of 8,000 characters before being passed to the language model, which explains why the mean context size of R1_k10 in Table 2.2 reaches the 8,000-character ceiling.

The final generation experiment used the Qwen2.5:3B language model, executed locally through

Ollama [5]. Local execution eliminated dependence on an external API daily-token quota during the final experiment.

The generated answers were compared with the benchmark reference answers using token-set lexical F1, computed as the harmonic mean of precision and recall between the lowercased alphanumeric token sets of the generated answer and the gold answer. Generation latency and context size were also recorded to evaluate efficiency. To determine whether differences in mean lexical F1 between configurations were statistically meaningful, a paired Wilcoxon signed-rank test was applied to the 60 per-question F1 scores for each pairwise comparison, since the same 60 questions were evaluated under all three configurations.

CHAPTER 4: RESULTS/ FINDINGS

4.1. Findings and Observations

The experimental evaluation of the Retrieval-Augmented Generation (RAG) system was conducted to examine the effects of retrieval depth and context optimization on retrieval performance, answer quality, and generation efficiency. The major findings and observations obtained from the experiments are summarized below:

- **Retrieval Performance:** Retrieval effectiveness improved consistently as the retrieval depth increased from $k=3$ to $k=20$. $\text{Hit}@k$ increased from 61.18% at $k=3$ to 82.33% at $k=20$, while MRR and nDCG also showed improvement.
- **Generation Performance:** Among the three configurations, R1_k10 achieved the highest mean lexical F1 score of 0.3136, compared with 0.3030 for R0_k3 and 0.3046 for R2_k10_optimized.
- **Generation Efficiency:** The R2_k10_optimized configuration achieved the lowest mean generation latency of 12.098 seconds, compared with 16.958 seconds for R1_k10. This shows that removing redundant context can reduce generation time.
- **Effect of Context Optimization:** Context optimization produced a small reduction in lexical F1 from 0.3136 to 0.3046 compared with the unoptimized top-10 configuration. The difference was statistically significant according to the Wilcoxon signed-rank test ($p = 0.005$).
- **Overall Observation:** The results indicate that increasing retrieval depth improves the availability and ranking of relevant information, while context optimization improves generation efficiency by reducing redundant context. However, the efficiency improvement is accompanied by a small reduction in lexical answer overlap.

4.2. Result Analysis

4.2.1. Quantitative Analysis

Retrieval Performance at Different k Values

Table 4.1: Retrieval Performance at Different Retrieval Depths

Retrieval Depth	Hit@k	MRR	nDCG
k = 3	0.6118	0.5061	0.5333
k = 5	0.6893	0.5239	0.5653
k = 10	0.7718	0.5349	0.5920

k = 20	0.8233	0.5385	0.6051
---------------	--------	--------	--------

The retrieval results demonstrate that increasing the retrieval depth consistently improved retrieval performance.

- At k=3, Hit@3 was 61.18%.
- At k=5, Hit@5 increased to 68.93%.
- At k=10, Hit@10 increased to 77.18%.
- At k=20, Hit@20 reached 82.33%.

Similarly, MRR increased from 0.5061 at k=3 to 0.5385 at k=20, while nDCG increased from 0.5333 to 0.6051.

These results indicate that increasing retrieval depth improves the probability of retrieving relevant information and also improves the overall ranking quality.

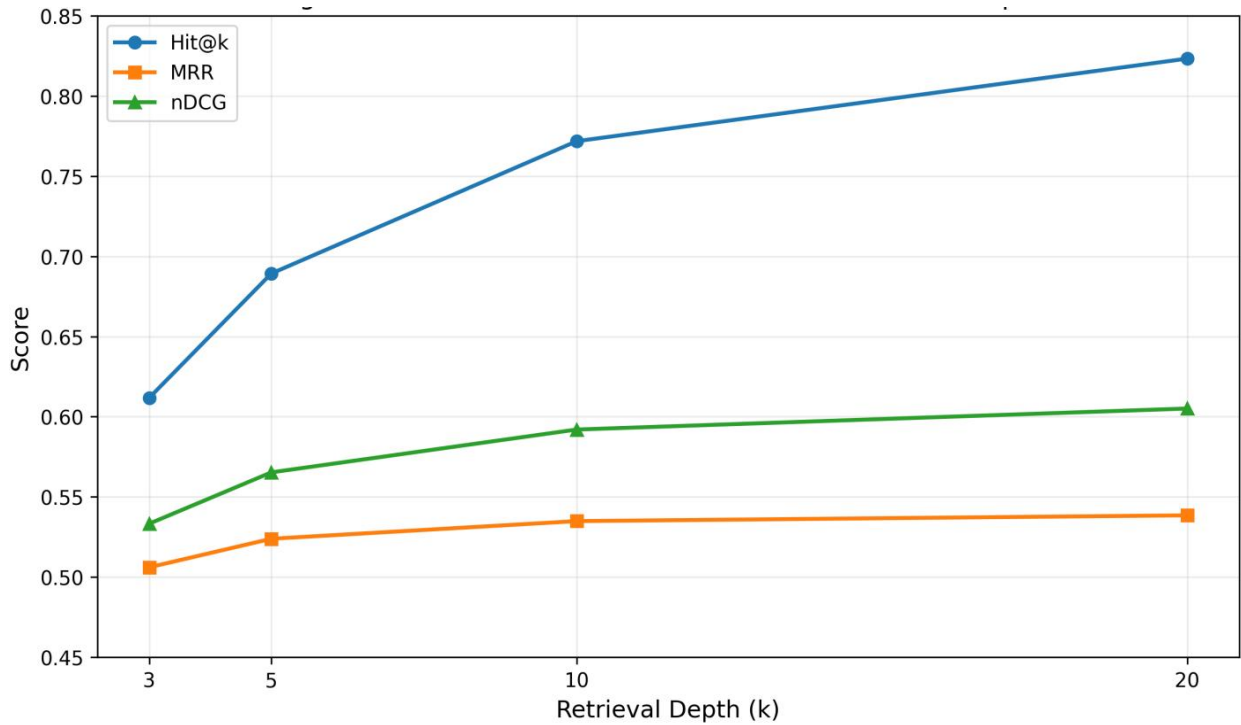


Figure 4.1.1: Retrieval Performance at Different Retrieval Depths

Generation Latency Analysis

The generation latency showed a substantial difference between the three configurations.

- **R0_k3:** 185.600 seconds
- **R1_k10:** 16.958 seconds
- **R2_k10_optimized:** 12.098 seconds

The optimized configuration achieved the lowest average latency. Compared with R1_k10, R2_k10_optimized reduced average latency by approximately 4.86 seconds. The substantially

higher latency observed for R0_k3 indicates that generation latency was affected by the local inference environment and runtime behavior in addition to the amount of retrieved context. This demonstrates that reducing unnecessary context can improve generation efficiency.

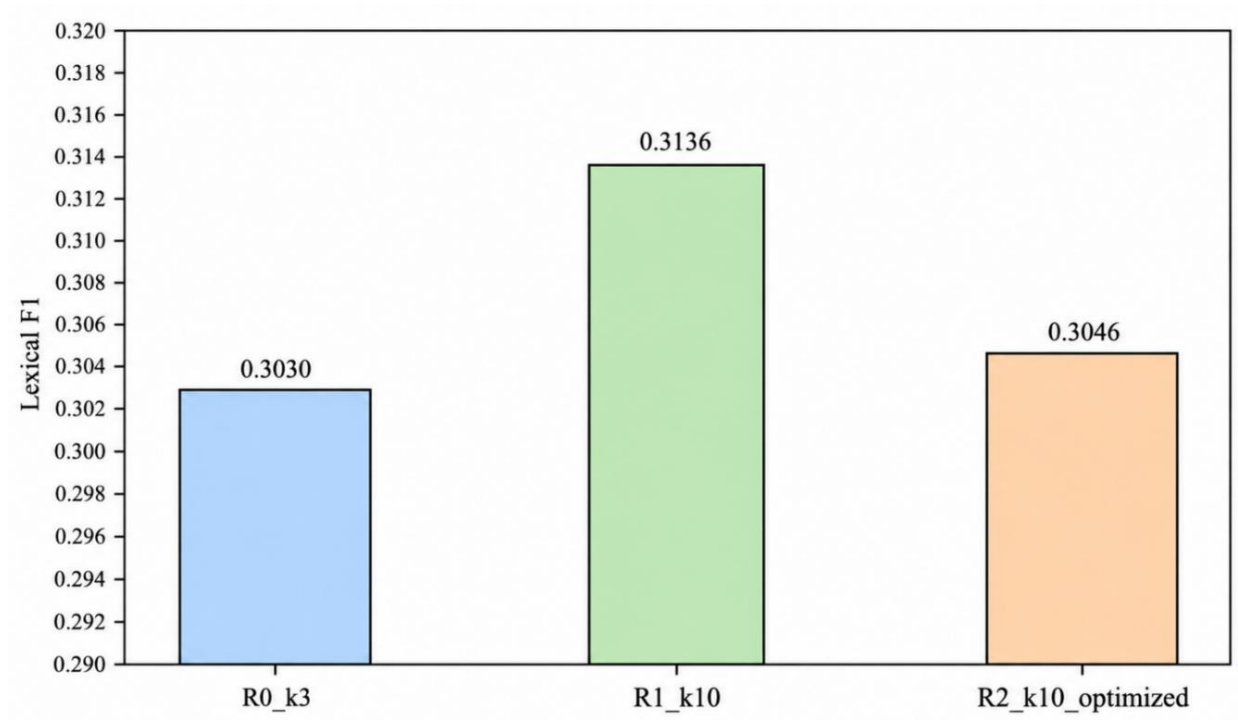


Figure 4.1.2: Lexical F1 Comparison of RAG Generation Configurations

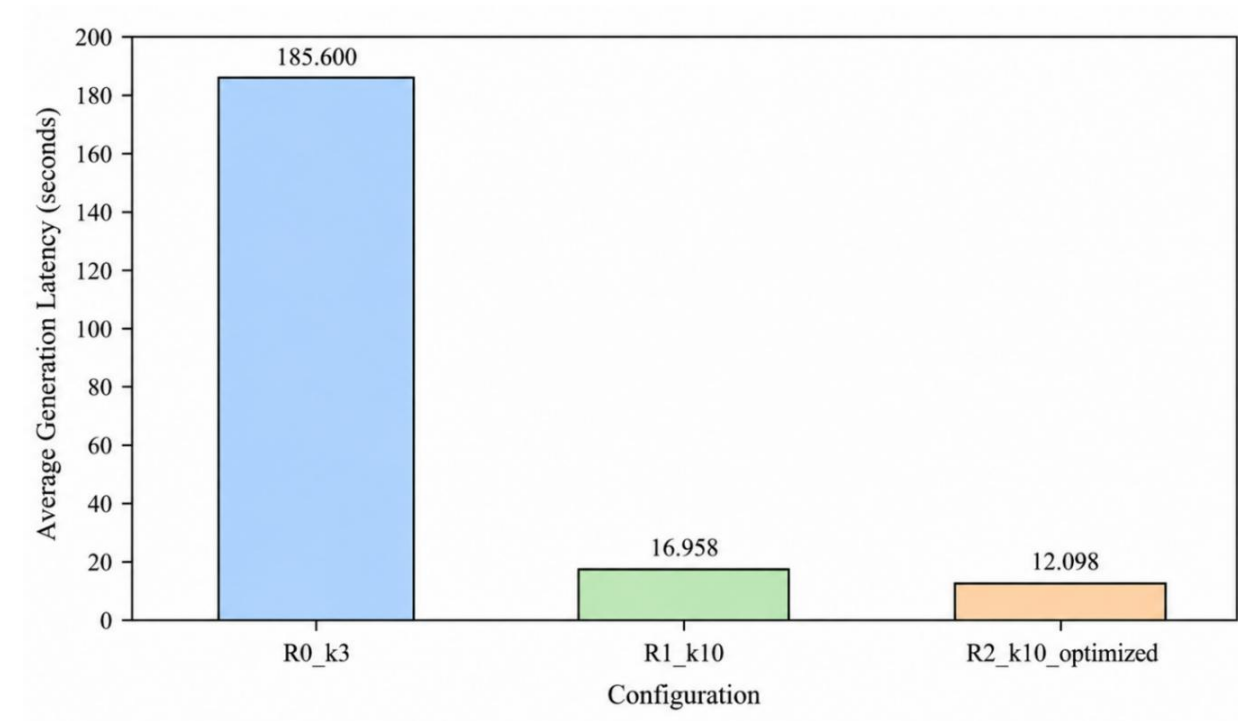


Figure 4.1.3: Average Generation Latency Across RAG Configurations

Table 4.1.2: Generation Performance of Different RAG Configurations

Metric	R0_k3	R1_k10	R2_k10_optimized
Questions	60	60	60
Successful	60	60	60
Errors	0	0	0
Mean Lexical F1	0.3030	0.3136	0.3046
IDK	23.33%	23.33%	25.00%
Mean Latency (s)	185.600	16.958	12.098
Mean Context Sections	2.45	3.23	3.10
Mean Context Characters	7014	8000	7862

Paired Lexical-F1 Difference

Table 4.1.3: Paired Lexical-F1 Differences and Wilcoxon Signed-Rank Test Results

Comparison	Mean $\Delta F1$	Wilcoxon p-value	Paired t-test p-value	Significant ($\alpha = 0.05$)
R0_k3 $\rightarrow$ R1_k10	+0.0106	0.263	0.133	No
R1_k10 $\rightarrow$ R2_k10_optimized	-0.0090	0.005	0.018	Yes
R0_k3 $\rightarrow$ R2_k10_optimized	+0.0016	0.942	0.801	No

A Wilcoxon signed-rank test was applied to the paired per-question lexical F1 scores ($n = 60$) for each of the three comparisons, since the same 60 questions were evaluated under every configuration. The comparison shows that increasing retrieval depth from 3 to 10 sections produced the largest positive change in mean lexical F1 (+0.0106), but this difference was not statistically significant (Wilcoxon $p = 0.263$). Context optimization subsequently produced a small mean reduction of -0.0090 relative to R1_k10, and this decrease was statistically significant (Wilcoxon $p = 0.005$; paired t-test $p = 0.018$). The overall difference between the

optimized configuration and the R0_k3 baseline was negligible and not significant (+0.0016, $p = 0.942$).

These results indicate that while R2_k10_optimized remained slightly above the R0_k3 baseline in raw mean F1, the reduction in F1 relative to the unoptimized R1_k10 configuration is unlikely to be due to chance. In other words, the latency improvement gained through context optimization came with a small but statistically measurable cost in lexical answer overlap compared with using the full top-10 context.

4.2.2. Qualitative Analysis (Actual System Output)

The implemented RAG system accepts document-based questions and retrieves relevant sections before generating an answer. The final generation experiment was performed using the locally hosted Qwen2.5 3B model through Ollama.

A sample output from the system can be presented in this section to demonstrate the complete question-answering process.

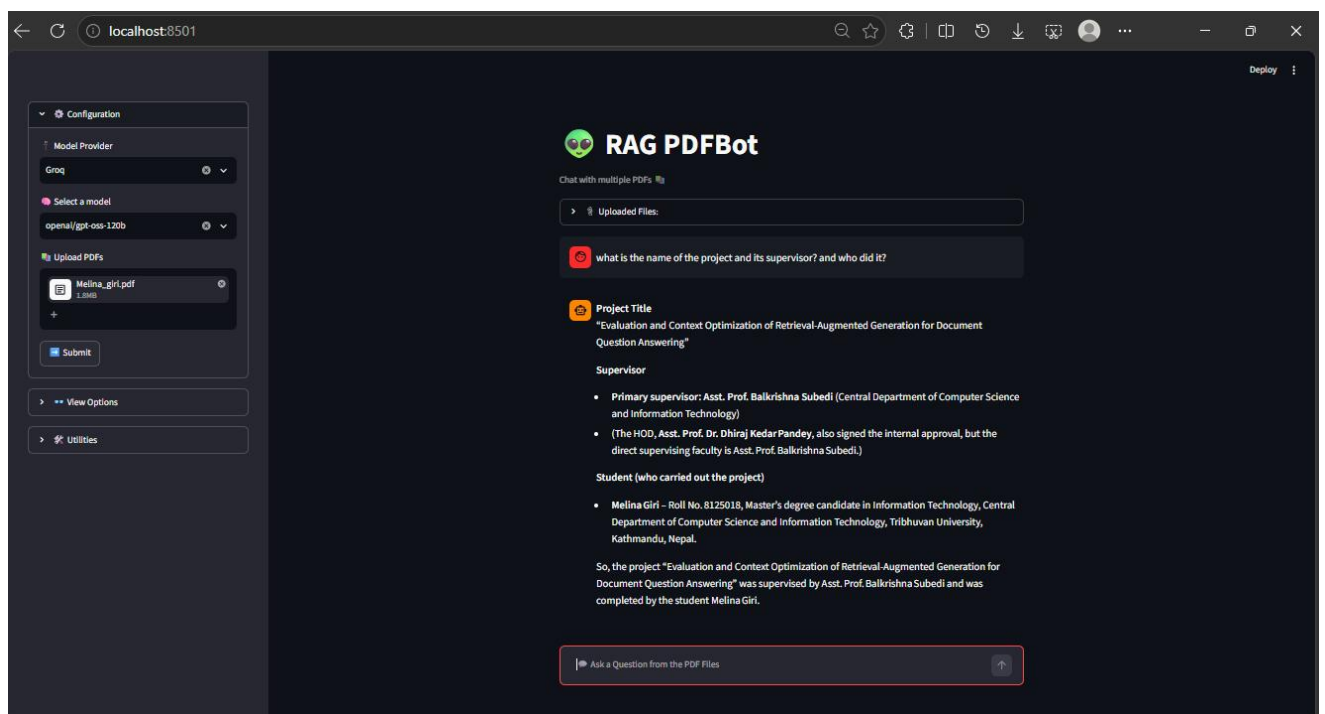


Figure 4.1.4: Sample Question and Generated Answer from the RAG System

Figure 2.5 presents an example interaction with the project's general-purpose interactive RAG interface, which supports multiple language-model providers (shown here using Groq); this interface is distinct from the fixed benchmark pipeline (Qwen2.5:3B via Ollama) used to produce the results in Tables 2.1–2.3. The example nonetheless demonstrates the practical operation of the underlying RAG pipeline, from question input through retrieval to final answer generation.

The qualitative output demonstrates the complete RAG pipeline:

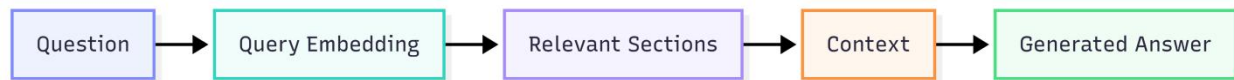


Figure 4.1.5: RAG Pipeline

The generated response is produced using the retrieved document context supplied to the language model as part of the RAG pipeline [1].

Analysis of the Three Configurations

R0_k3: The system uses only the top three retrieved sections. This provides a relatively small context but may omit information required for answering complex questions.

R1_k10: The system retrieves ten sections, providing more information to the language model. This configuration achieved the highest lexical F1 score (**0.3136**) among the three generation configurations.

R2_k10_optimized: The system initially retrieves ten sections and then removes redundant sections using Jaccard word-set overlap (≥ 0.70), retaining at most five sections. Although its lexical F1 (0.3046) was slightly lower than R1_k10, a difference that was also statistically significant, it achieved the lowest generation latency (12.098 seconds), demonstrating a clear efficiency advantage.

Overall, the experimental results suggest that retrieval depth has a positive, though not statistically significant, effect on generation-level answer overlap in this sample, while context optimization can improve generation efficiency but produces a small, statistically significant reduction in lexical F1 relative to the unoptimized top-10 configuration. The lexical F1 results should be interpreted as an automatic word-overlap measure rather than definitive semantic correctness.

CHAPTER 5: CONCLUSION

5.1. Conclusion

This research successfully implemented and evaluated a Retrieval-Augmented Generation (RAG) system for document question answering based on the retrieval-and-generation framework described in previous RAG research [1]. The study demonstrated how combining semantic information retrieval with a language model can provide answers based on relevant sections retrieved from a document collection rather than relying solely on the language model's internal knowledge.

Through a systematic methodology involving benchmark document loading, embedding generation, vector-based retrieval, context selection, and answer generation, the system was able to retrieve relevant document sections and use them as context for generating answers. The retrieval component was evaluated using different retrieval depths, including $k=3$, $k=5$, $k=10$, and $k=20$.

The retrieval results showed that increasing the number of retrieved sections generally improved retrieval effectiveness. $\text{Hit}@k$ increased from 61.18% at $k=3$ to 82.33% at $k=20$, while MRR increased from 0.5061 to 0.5385. Similarly, nDCG increased from 0.5333 to 0.6051, indicating improved ranking quality as retrieval depth increased.

The generation experiment compared three configurations: $R0_k3$, $R1_k10$, and $R2_k10_optimized$. Among these configurations, $R1_k10$ achieved the highest mean lexical F1 score of 0.3136, compared with 0.3030 for $R0_k3$ and 0.3046 for $R2_k10_optimized$. A paired Wilcoxon signed-rank test on the 60 per-question F1 scores showed that the reduction in F1 from $R1_k10$ to $R2_k10_optimized$ was statistically significant ($p = 0.005$), while the differences involving $R0_k3$ were not statistically significant. Context optimization reduced average latency from 16.958 seconds to 12.098 seconds.

The final experiment successfully generated answers for 60 questions under all three configurations, resulting in 180 successful generations without errors. The results demonstrate that increasing retrieval depth can improve the availability of relevant information, while context optimization can meaningfully reduce unnecessary context and generation time, at the cost of a small, statistically significant reduction in lexical answer overlap relative to the unoptimized top-10 configuration.

In conclusion, the study validates that retrieval depth and context selection are important factors in RAG-based document question answering. The findings show that providing additional relevant context can improve retrieval and slightly improve lexical answer overlap,

while context optimization provides genuine efficiency benefits alongside a measurable trade-off in answer overlap. However, lexical F1 should be interpreted as an automatic word-overlap measure rather than definitive semantic answer correctness.

5.2. Future Recommendation

Based on the findings of this study, the following areas are recommended for future research and improvement:

1. Expand the Generation Evaluation Dataset

The current generation experiment evaluates 60 questions from the benchmark, while the retrieval evaluation uses the complete set of 3,045 queries. Future work can evaluate generation on a larger number of questions to obtain more statistically robust results, increase the statistical power of paired significance tests, and reduce the effect of sample selection.

2. Improve Context Optimization

The current optimization method removes sections based on Jaccard word-set redundancy. Future work can investigate more advanced context-selection methods based on semantic similarity, relevance scores, diversity, and learned ranking techniques. This may reduce unnecessary context while preserving information required for accurate answers.

3. Evaluate Different Embedding Models

The current system uses all-MiniLM-L12-v2 for document and query embeddings. Future studies can compare other embedding models to determine whether stronger semantic representations improve retrieval performance, particularly for complex and domain-specific questions.

4. Evaluate Different Language Models

The final experiment uses the locally executed Qwen2.5 3B model. Future research can compare different open-weight and hosted language models to determine how model capability affects answer quality, hallucination, latency, and resource requirements.

5. Use Semantic Answer Evaluation

The current generation evaluation primarily uses lexical F1, which measures word overlap between generated and reference answers. Future work should incorporate semantic evaluation methods such as BERTScore, embedding-based similarity, LLM-as-a-judge evaluation, and human evaluation to better measure whether generated answers are actually correct.

6. Improve Retrieval and Ranking

Although increasing k improved retrieval coverage, retrieving additional sections may

also introduce irrelevant information. Future work can investigate reranking models, hybrid keyword-plus-dense retrieval, and query expansion to improve the relevance of retrieved context.

7. Integration into Real Applications

Beyond benchmark evaluation, the RAG system can be deployed as a practical document question-answering application for research papers, educational materials, organizational documents, or other specialized document collections. Evaluating response time, user satisfaction, answer correctness, and resource consumption in real-world environments would provide further practical validation.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Y. Wang, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [4] Vectara, “Open RAG Benchmark,” Vectara Research and Benchmark Dataset.
- [5] Qwen Team, “Qwen2.5: A Party of Foundation Models,” 2024.
- [6] Sentence-Transformers, “all-MiniLM-L12-v2,” Hugging Face Model Card.

Appendix

A.1 Main Interface Code

The code in Sections A.1–A.4 is drawn from the project's general-purpose interactive interface (supporting Groq/Gemini as configurable providers), separate from the fixed benchmark pipeline described in Section 2.4 that generated the reported results. The benchmark pipeline code is shown in Section A.5. This section shows the main interface of the RAG-based document question-answering system.

```
client > app.py x
1 import streamlit as st
2
3 from state.session import setup_session_state, is_chat_ready
4 from components.chat import (
5     render_chat_history,
6     render_download_chat_history,
7     render_uploaded_files_expander,
8     render_user_input
9 )
10 from components.sidebar import (
11     render_model_selector,
12     render_view_selector,
13     sidebar_file_upload,
14     sidebar_provider_change_check,
15     sidebar_utilities
16 )
17 from components.inspector import render_inspect_query
18
19
20 def main():
21     st.set_page_config(page_title="RAG PDFBot", layout="centered")
22     st.title("🤖 RAG PDFBot")
23     st.caption("Chat with multiple PDFs :books:")
24
25     setup_session_state()
26
27     if st.session_state.get("chat_history"):
28         render_download_chat_history()
29
30     with st.sidebar:
31         with st.expander("⚙️ Configuration", expanded=True):
32             model_provider, model = render_model_selector()
33             sidebar_file_upload(model_provider)
34             sidebar_provider_change_check(model_provider, model)
35
36         view_option = render_view_selector()
37         sidebar_utilities()
38
39     if not st.session_state.get(f"uploaded_files_{st.session_state.uploader_key}", []):
40         st.info("📄 Please upload and submit PDFs to start chatting.")
41
42     if st.session_state.get("unsubmitted_files", False):
43         st.warning("📄 New PDFs uploaded. Please submit before chatting.")
44
45     if st.session_state.get("chat_ready") and st.session_state.get("pdf_files", []):
46         render_uploaded_files_expander()
47
48     if view_option == "💬 Chat":
49         if st.session_state.get("chat_history", []):
50             render_chat_history()
51
52         if is_chat_ready():
53             render_user_input(model_provider, model)
54     elif view_option == "🔍 Inspector":
55         if is_chat_ready():
56             render_inspect_query(model_provider)
57
58 if __name__ == "__main__":
59     main()
60
```

Figure A.1: Main Interface Code

A.2 Loading Documents and Corpus Sections

This section demonstrates how the benchmark corpus is loaded and its pre-segmented document sections are prepared for embedding generation and retrieval. The document-processing pipeline includes:

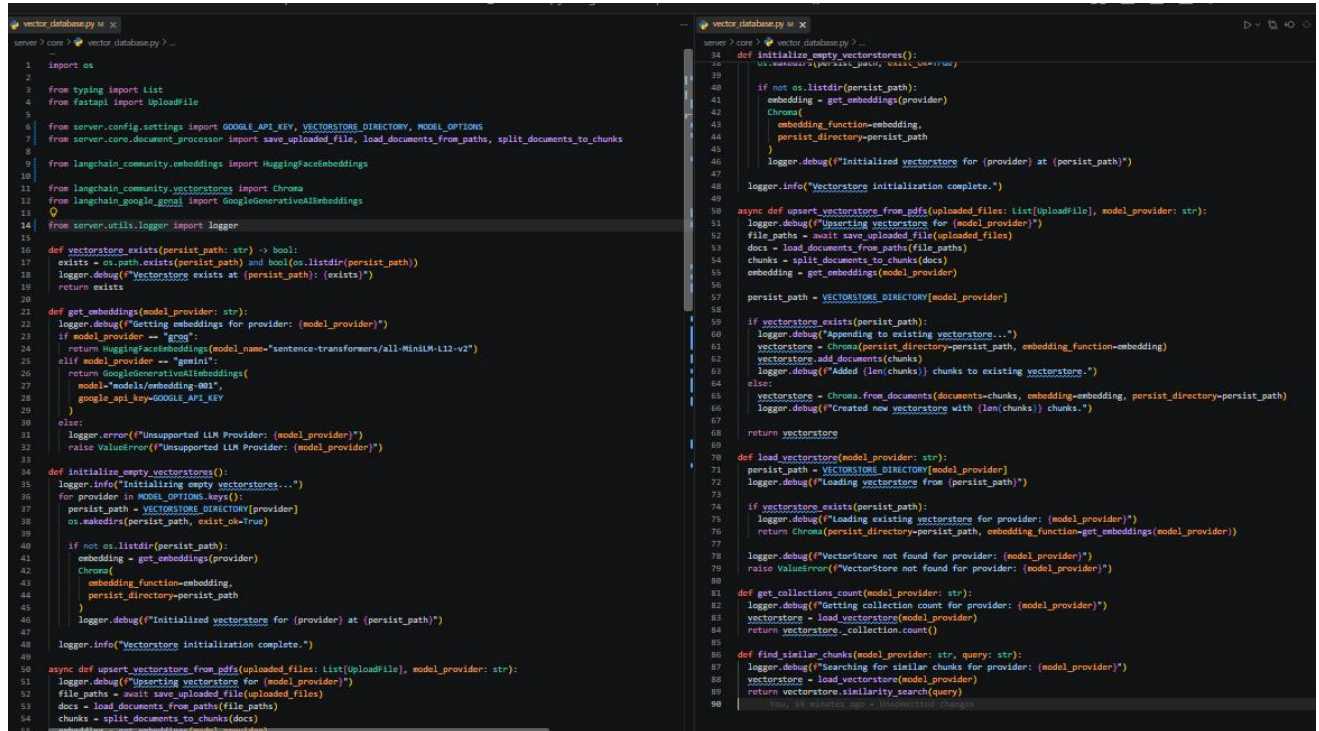
- Loading the benchmark corpus JSON files
- Reading each paper’s pre-segmented sections
- Preparing document and section metadata
- Generating embeddings

```
document_processor.py 1/1 X
server > code > document_processor.py > ...
You, 54 minutes ago | 2 authors (dash05 and one other)
1 import os
2 import aiofiles
3
4 from typing import List
5 from fastapi import UploadFile
6
7 from langchain_community.document_loaders import PyPDFLoader
8 from langchain_text_splitters import TokenTextSplitter
9
10 from server.config.settings import TEMPFILE_UPLOAD_DIRECTORY
11 from server.utils.logger import logger
12
13 def validate_pdf(file: UploadFile, max_size_mb: int = 200):
14     if not file.filename.endswith(".pdf"):
15         logger.warning(f"Invalid file type: {file.filename}")
16         raise ValueError(f"{file.filename} is not a valid PDF file.")
17
18     file_size_mb = len(file.file.read()) / (1024 * 1024)
19     file.file.seek(0)
20
21     if file_size_mb > max_size_mb:
22         logger.warning(f"File too large: {file.filename} ({file_size_mb:.2f} MB)")
23         raise ValueError(f"PDF file size exceeds the maximum allowed size of {max_size_mb} MB.")
24
25     logger.debug(f"Validated PDF: {file.filename} ({file_size_mb:.2f} MB)")
26
27 async def save_uploaded_file(files: List[UploadFile]) -> List[str]:
28     os.makedirs(TEMPFILE_UPLOAD_DIRECTORY, exist_ok=True)
29     file_paths = []
30
31     for file in files:
32         validate_pdf(file)
33         file_path = os.path.join(TEMPFILE_UPLOAD_DIRECTORY, file.filename)
34         async with aiofiles.open(file_path, "wb") as f:
35             content = await file.read()
36             await f.write(content)
37         file_paths.append(file_path)
38         logger.debug(f"Saved uploaded file: {file.filename} to {file_path}")
39
40     return file_paths
41
42 def load_documents_from_paths(file_paths: List[str]):
43     docs = []
44     for file_path in file_paths:
45         loader = PyPDFLoader(file_path)
46         loaded = loader.load()
47         logger.debug(f"Loaded {len(loaded)} documents from {file_path}")
48         docs.extend(loaded)
49
50     return docs
51
52 def split_documents_to_chunks(docs) -> List[str]:
53     text_splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
54     chunks = text_splitter.split_documents(docs)
55     logger.debug(f"Split {len(docs)} docs into {len(chunks)} chunks")
56     return chunks
57
You, 54 minutes ago • Uncommitted changes
```

Figure A.2: Corpus Loading and Section Preparation Code

A.3 Embedding Generation and Vector Retrieval

The following code implements the generation of dense vector representations using the all-MiniLM-L12-v2 embedding model [6]. The generated vectors are used to calculate similarity between the query and document sections.



```
vector_database.py
1 import os
2
3 from typing import List
4 from fastapi import UploadFile
5
6 from server.config.settings import GOOGLE_API_KEY, VECTORSTORE_DIRECTORY, MODEL_OPTIONS
7 from server.core.document_processor import save_uploaded_file, load_documents_from_paths, split_documents_to_chunks
8
9 from langchain_community.embeddings import HuggingFaceEmbeddings
10
11 from langchain_community.vectorstores import Chroma
12 from langchain_google_genai import GoogleGenerativeAIEmbeddings
13
14 from server.utils.logger import logger
15
16 def vectorstore_exists(persist_path: str) -> bool:
17     exists = os.path.exists(persist_path) and bool(os.listdir(persist_path))
18     logger.debug(f"Vectorstore exists at {persist_path}: {exists}")
19     return exists
20
21 def get_embeddings(model_provider: str):
22     logger.debug(f"Getting embeddings for provider: {model_provider}")
23     if model_provider == "gpt4":
24         return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L12-v2")
25     elif model_provider == "gemini":
26         return GoogleGenerativeAIEmbeddings(
27             model="models/embedding-001",
28             google_api_key=GOOGLE_API_KEY
29         )
30     else:
31         logger.error(f"Unsupported LLM Provider: {model_provider}")
32         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
33
34 def initialize_empty_vectorstores():
35     logger.info("Initializing empty vectorstores...")
36     for provider in MODEL_OPTIONS.keys():
37         persist_path = VECTORSTORE_DIRECTORY[provider]
38         os.makedirs(persist_path, exist_ok=True)
39
40         if not os.listdir(persist_path):
41             embedding = get_embeddings(provider)
42             Chroma(
43                 embedding_function=embedding,
44                 persist_directory=persist_path
45             )
46             logger.debug(f"Initialized vectorstore for {provider} at {persist_path}")
47
48     logger.info("Vectorstore initialization complete.")
49
50 async def upsert_vectorstore_from_pdf(uploaded_files: List[UploadFile], model_provider: str):
51     logger.debug(f"Upserting vectorstore for {model_provider}")
52     file_paths = await save_uploaded_files(uploaded_files)
53     docs = load_documents_from_paths(file_paths)
54     chunks = split_documents_to_chunks(docs)
55     embedding = get_embeddings(model_provider)
56
57     persist_path = VECTORSTORE_DIRECTORY[model_provider]
58
59     if vectorstore_exists(persist_path):
60         logger.debug("Appending to existing vectorstore...")
61         vectorstore = Chroma(persist_directory=persist_path, embedding_function=embedding)
62         vectorstore.add_documents(chunks)
63         logger.debug(f"Added {len(chunks)} chunks to existing vectorstore.")
64     else:
65         vectorstore = Chroma.from_documents(docs, embedding=embedding, persist_directory=persist_path)
66         logger.debug(f"Created new vectorstore with {len(chunks)} chunks.")
67
68     return vectorstore
69
70 def load_vectorstore(model_provider: str):
71     persist_path = VECTORSTORE_DIRECTORY[model_provider]
72     logger.debug(f"Loading vectorstore from {persist_path}")
73
74     if vectorstore_exists(persist_path):
75         logger.debug(f"Loading existing vectorstore for provider: {model_provider}")
76         return Chroma(persist_directory=persist_path, embedding_function=get_embeddings(model_provider))
77
78     logger.debug(f"Vectorstore not found for provider: {model_provider}")
79     raise ValueError(f"Vectorstore not found for provider: {model_provider}")
80
81 def get_collections_count(model_provider: str):
82     logger.debug(f"Getting collection count for provider: {model_provider}")
83     vectorstore = load_vectorstore(model_provider)
84     return vectorstore.collection.count()
85
86 def find_similar_chunks(model_provider: str, query: str):
87     logger.debug(f"Searching for similar chunks for provider: {model_provider}")
88     vectorstore = load_vectorstore(model_provider)
89     return vectorstore.similarity_search(query)
90
```

```
vector_store.py
1 def initialize_empty_vectorstores():
2     os.makedirs(persist_path, exist_ok=True)
3
4     if not os.listdir(persist_path):
5         embedding = get_embeddings(provider)
6         Chroma(
7             embedding_function=embedding,
8             persist_directory=persist_path
9         )
10        logger.debug(f"Initialized vectorstore for {provider} at {persist_path}")
11
12    logger.info("Vectorstore initialization complete.")
13
14    async def upsert_vectorstore_from_pdf(uploaded_files: List[UploadFile], model_provider: str):
15        logger.debug(f"Upserting vectorstore for {model_provider}")
16        file_paths = await save_uploaded_files(uploaded_files)
17        docs = load_documents_from_paths(file_paths)
18        chunks = split_documents_to_chunks(docs)
19        embedding = get_embeddings(model_provider)
20
21        persist_path = VECTORSTORE_DIRECTORY[model_provider]
22
23        if vectorstore_exists(persist_path):
24            logger.debug("Appending to existing vectorstore...")
25            vectorstore = Chroma(persist_directory=persist_path, embedding_function=embedding)
26            vectorstore.add_documents(chunks)
27            logger.debug(f"Added {len(chunks)} chunks to existing vectorstore.")
28        else:
29            vectorstore = Chroma.from_documents(docs, embedding=embedding, persist_directory=persist_path)
30            logger.debug(f"Created new vectorstore with {len(chunks)} chunks.")
31
32        return vectorstore
33
34    def load_vectorstore(model_provider: str):
35        persist_path = VECTORSTORE_DIRECTORY[model_provider]
36        logger.debug(f"Loading vectorstore from {persist_path}")
37
38        if vectorstore_exists(persist_path):
39            logger.debug(f"Loading existing vectorstore for provider: {model_provider}")
40            return Chroma(persist_directory=persist_path, embedding_function=get_embeddings(model_provider))
41
42        logger.debug(f"Vectorstore not found for provider: {model_provider}")
43        raise ValueError(f"Vectorstore not found for provider: {model_provider}")
44
45    def get_collections_count(model_provider: str):
46        logger.debug(f"Getting collection count for provider: {model_provider}")
47        vectorstore = load_vectorstore(model_provider)
48        return vectorstore.collection.count()
49
50    def find_similar_chunks(model_provider: str, query: str):
51        logger.debug(f"Searching for similar chunks for provider: {model_provider}")
52        vectorstore = load_vectorstore(model_provider)
53        return vectorstore.similarity_search(query)
54
```

Figure A.3: Embedding Generation and Similarity-Based Retrieval Code

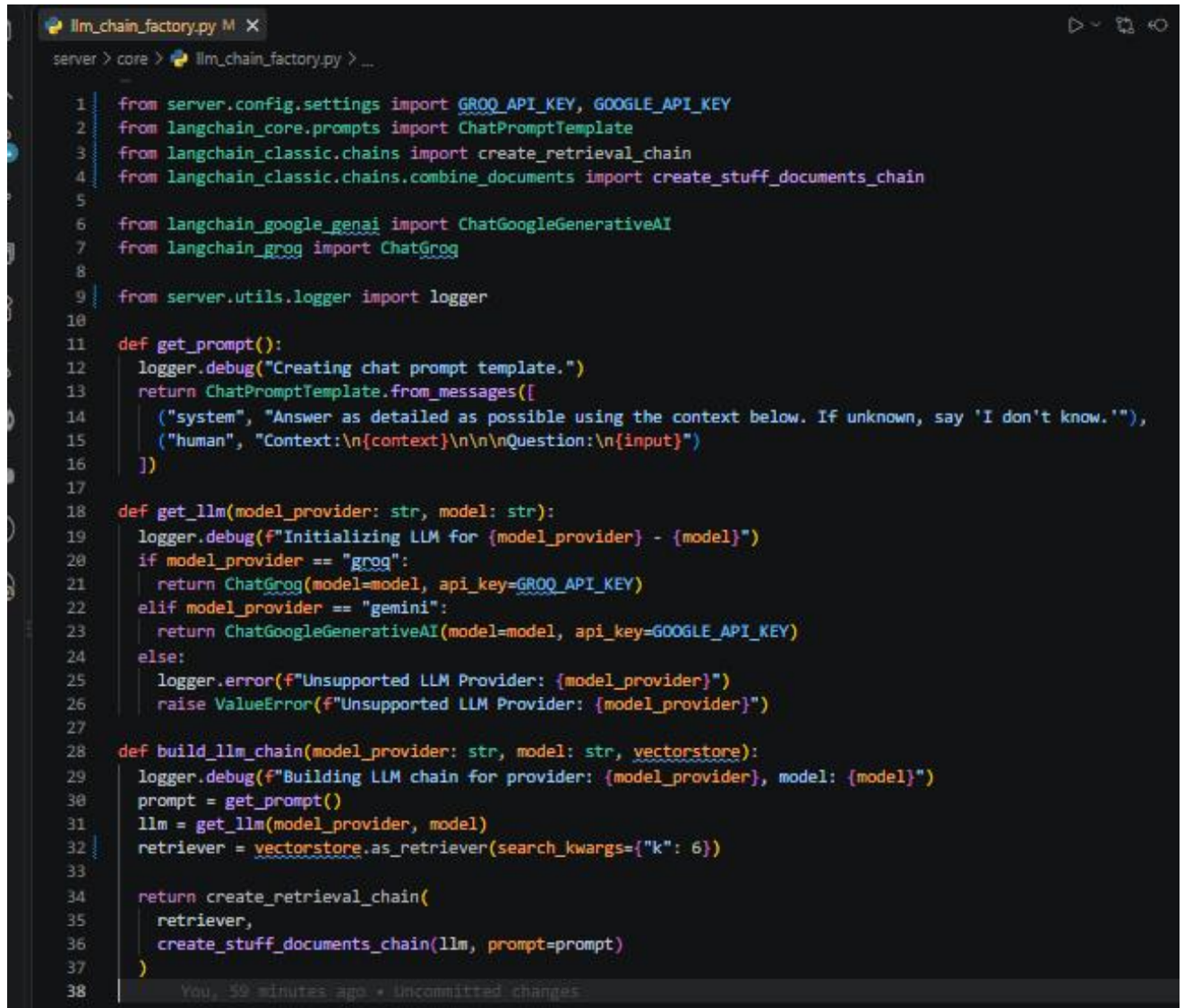
A.4 RAG Answer Generation Code

This code demonstrates how the retrieved document context is combined with the user's question and passed to the language model.

The system follows the sequence:

User Question → Retrieved Sections → (Optional) Context Optimization → Character-Limited Context → Prompt → Language Model → Generated Answer

The final generation experiment used the locally executed Qwen2.5:3B model through Ollama.



```
llm_chain_factory.py M X
server > core > llm_chain_factory.py > ...

1  from server.config.settings import GROQ_API_KEY, GOOGLE_API_KEY
2  from langchain_core.prompts import ChatPromptTemplate
3  from langchain_classic.chains import create_retrieval_chain
4  from langchain_classic.chains.combine_documents import create_stuff_documents_chain
5
6  from langchain_google_genai import ChatGoogleGenerativeAI
7  from langchain_groq import ChatGroq
8
9  from server.utils.logger import logger
10
11 def get_prompt():
12     logger.debug("Creating chat prompt template.")
13     return ChatPromptTemplate.from_messages([
14         ("system", "Answer as detailed as possible using the context below. If unknown, say 'I don't know.'"),
15         ("human", "Context:\n{context}\n\nQuestion:\n{input}")
16     ])
17
18 def get_llm(model_provider: str, model: str):
19     logger.debug(f"Initializing LLM for {model_provider} - {model}")
20     if model_provider == "groq":
21         return ChatGroq(model=model, api_key=GROQ_API_KEY)
22     elif model_provider == "gemini":
23         return ChatGoogleGenerativeAI(model=model, api_key=GOOGLE_API_KEY)
24     else:
25         logger.error(f"Unsupported LLM Provider: {model_provider}")
26         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
27
28 def build_llm_chain(model_provider: str, model: str, vectorstore):
29     logger.debug(f"Building LLM chain for provider: {model_provider}, model: {model}")
30     prompt = get_prompt()
31     llm = get_llm(model_provider, model)
32     retriever = vectorstore.as_retriever(search_kwargs={"k": 6})
33
34     return create_retrieval_chain(
35         retriever,
36         create_stuff_documents_chain(llm, prompt=prompt)
37     )
38
You, 59 minutes ago • Uncommitted changes
```

Figure A.4: RAG Answer Generation Code

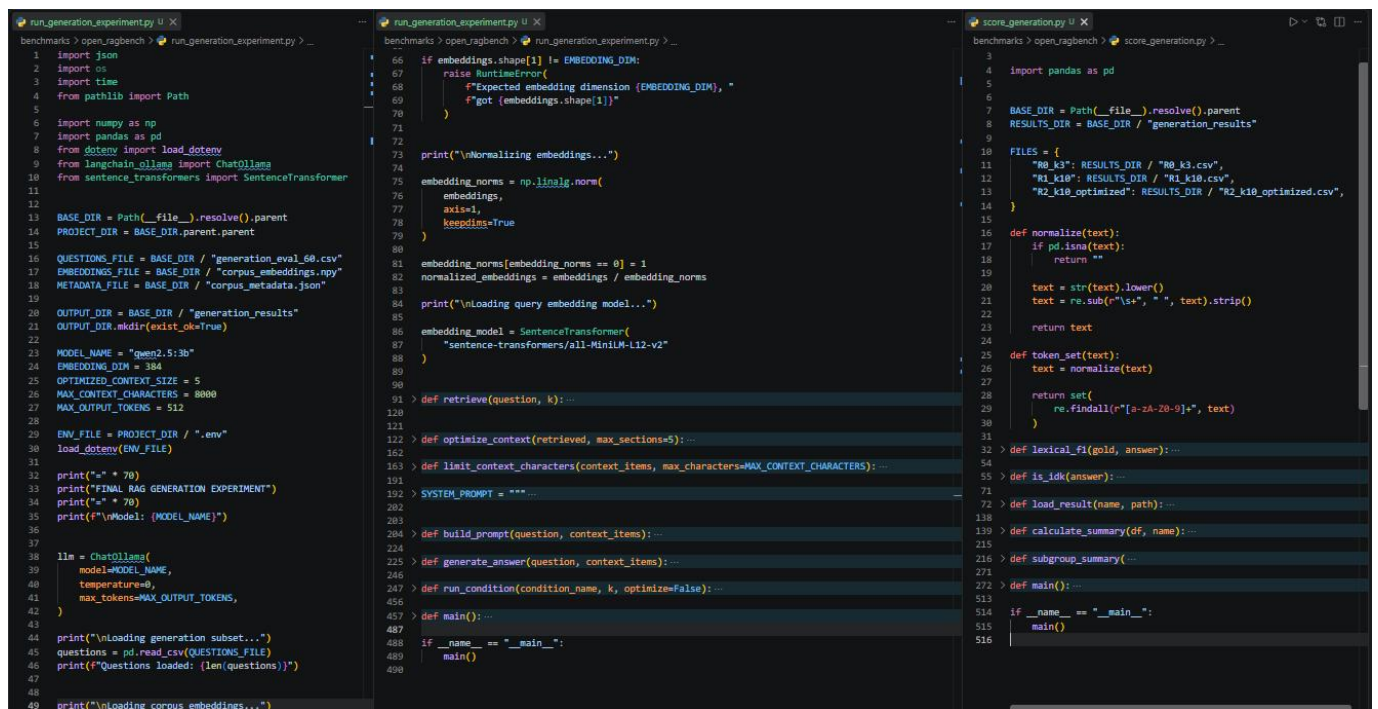
A.5 Evaluation and Result Analysis Code

This section contains the code used to evaluate the RAG system. Retrieval performance is evaluated using Hit@k, MRR, and nDCG, while generation performance is evaluated using lexical F1, IDK percentage, latency, and context size.

The generation experiment compares:

- R0_k3
- R1_k10
- R2_k10_optimized

The resulting evaluation tables are generated for overall performance, question type, source type, and paired lexical-F1 differences.



```
run_generation_experiment.py
1 import json
2 import os
3 import time
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 from dotenv import load_dotenv
9 from langchain_ollama import ChatOllama
10 from sentence_transformers import SentenceTransformer
11
12 BASE_DIR = Path(__file__).resolve().parent
13 PROJECT_DIR = BASE_DIR.parent.parent
14
15 QUESTIONS_FILE = BASE_DIR / "generation_eval_68.csv"
16 EMBEDDINGS_FILE = BASE_DIR / "corpus_embeddings.npy"
17 METADATA_FILE = BASE_DIR / "corpus_metadata.json"
18
19 OUTPUT_DIR = BASE_DIR / "generation_results"
20 OUTPUT_DIR.mkdir(exist_ok=True)
21
22 MODEL_NAME = "qwen2.5:3b"
23 EMBEDDING_DIM = 384
24 OPTIMIZED_CONTEXT_SIZE = 5
25 MAX_CONTEXT_CHARACTERS = 8000
26 MAX_OUTPUT_TOKENS = 512
27
28 ENV_FILE = PROJECT_DIR / ".env"
29 load_dotenv(ENV_FILE)
30
31 print(f"{'=' * 70}")
32 print("FINAL RAG GENERATION EXPERIMENT")
33 print(f"{'=' * 70}")
34 print(f"Model: {MODEL_NAME}")
35
36 llm = ChatOllama(
37     model=MODEL_NAME,
38     temperature=0,
39     max_tokens=MAX_OUTPUT_TOKENS,
40 )
41
42 print("Loading generation subset...")
43 questions = pd.read_csv(QUESTIONS_FILE)
44 print(f"Questions loaded: {len(questions)}")
45
46 print("Loading corpus embeddings...")
47
48 run_generation_experiment.py
49
50 if embeddings.shape[1] != EMBEDDING_DIM:
51     raise RuntimeError(
52         f"Expected embedding dimension {EMBEDDING_DIM}, "
53         f"got {embeddings.shape[1]}"
54     )
55
56 print("Normalizing embeddings...")
57
58 embedding_norms = np.linalg.norm(
59     embeddings,
60     axis=1,
61     keepdims=True
62 )
63
64 embedding_norms[embedding_norms == 0] = 1
65 normalized_embeddings = embeddings / embedding_norms
66
67 print("Loading query embedding model...")
68
69 embedding_model = SentenceTransformer(
70     "sentence-transformers/all-MiniLM-L12-v2"
71 )
72
73 def retrieve(question, k):
74     query_embedding = embedding_model.encode(question)
75     scores = query_embedding.dot(normalized_embeddings.T)
76     sorted_indices = np.argsort(-scores)
77     retrieved_items = []
78     for i in sorted_indices[:k]:
79         retrieved_items.append(i)
80     return retrieved_items
81
82 def optimize_context(retrieved, max_sections=5):
83     sections = []
84     for i in retrieved:
85         sections.append(i)
86         if len(sections) == max_sections:
87             break
88     return sections
89
90 def limit_context_characters(context_items, max_characters=MAX_CONTEXT_CHARACTERS):
91     limited_context = []
92     for item in context_items:
93         limited_context.append(item[:max_characters])
94     return limited_context
95
96 SYSTEM_PROMPT = """
97 You are a helpful assistant.
98 Answer the question based on the provided context.
99 If you cannot find the answer, say "I don't know".
100 """
101
102 def build_prompt(question, context_items):
103     prompt = SYSTEM_PROMPT + "\n"
104     for item in context_items:
105         prompt += item + "\n"
106     prompt += question + "\n"
107     return prompt
108
109 def generate_answer(question, context_items):
110     prompt = build_prompt(question, context_items)
111     answer = llm.predict(prompt)
112     return answer
113
114 def run_condition(condition_name, k, optimize=False):
115     retrieved = retrieve(condition_name, k)
116     if optimize:
117         retrieved = optimize_context(retrieved)
118     limited_context = limit_context_characters(retrieved)
119     answer = generate_answer(condition_name, limited_context)
120     return answer
121
122 def main():
123     run_condition("R0_k3", 3)
124     run_condition("R1_k10", 10)
125     run_condition("R2_k10_optimized", 10)
126
127 if __name__ == "__main__":
128     main()
129
130 score_generation.py
131
132 import pandas as pd
133
134 BASE_DIR = Path(__file__).resolve().parent
135 RESULTS_DIR = BASE_DIR / "generation_results"
136
137 FILES = {
138     "R0_k3": RESULTS_DIR / "R0_k3.csv",
139     "R1_k10": RESULTS_DIR / "R1_k10.csv",
140     "R2_k10_optimized": RESULTS_DIR / "R2_k10_optimized.csv",
141 }
142
143 def normalize(text):
144     if pd.isna(text):
145         return ""
146     text = str(text).lower()
147     text = re.sub(r"[^\w]", " ", text).strip()
148     return text
149
150 def token_set(text):
151     text = normalize(text)
152     return set(
153         re.findall(r"[a-zA-Z0-9]+", text)
154     )
155
156 def lexical_f1(gold, answer):
157     gold_tokens = token_set(gold)
158     answer_tokens = token_set(answer)
159     intersection = len(gold_tokens & answer_tokens)
160     union = len(gold_tokens | answer_tokens)
161     return intersection / union
162
163 def is_idk(answer):
164     return "I don't know" in answer.lower()
165
166 def load_result(name, path):
167     df = pd.read_csv(path)
168     return df
169
170 def calculate_summary(df, name):
171     df["lexical_f1"] = df["gold"].apply(
172         lambda gold: lexical_f1(gold, df["answer"])
173     )
174     df["is_idk"] = df["answer"].apply(is_idk)
175     return df
176
177 def subgroup_summary(df, name):
178     df = calculate_summary(df, name)
179     summary = df.groupby("question_type").agg(
180         lexical_f1_mean=("lexical_f1", "mean"),
181         is_idk_mean=("is_idk", "mean"),
182     )
183     return summary
184
185 def main():
186     for name, path in FILES.items():
187         df = load_result(name, path)
188         df = calculate_summary(df, name)
189         subgroup_summary(df, name)
190
191 if __name__ == "__main__":
192     main()
193
```

Figure A.5: RAG Evaluation and Result Analysis Code